

Report: Network Packet Sniffer

Name: Faizan Ali

ID: 52906

Course: Information Security

Time Slot: Wednesday/Thursday

Network Packet Sniffer Project Report

1. Project Title Network Packet Sniffer with Real-Time Analysis and Deep Packet Inspection

2. Abstract This project focuses on developing a comprehensive Network Packet Sniffer that enables users to capture, analyze, and inspect network packets in real time. The application is equipped with features such as protocol dissection, payload extraction, suspicious pattern detection, and traffic flow analysis. Designed for educational and practical use, the tool provides insights into network communication and assists in identifying anomalies, making it a valuable resource for network monitoring and troubleshooting.

3. Introduction Network packet sniffing is a critical technique used in network administration, troubleshooting, and security analysis. This project aims to provide a user-friendly interface for packet capture and analysis, helping users understand network protocols and detect potential issues. The project includes advanced features such as real-time packet analysis, deep packet inspection, and the ability to export captured data for future analysis.

4. Objectives

- To develop a user-friendly packet sniffing application.
- To enable real-time packet capture and analysis.
- To implement advanced protocol parsing for HTTP, SSL/TLS, and custom protocols.
- To detect and alert users about suspicious network patterns like port scanning and DDoS attacks.
- To provide functionality for exporting packet data for offline analysis.

5. Methodology

5.1 Tools and Technologies

- **Programming Language:** Python
- **IDE:** PyCharm
- **Libraries:**
 - scapy for packet capture and analysis
 - tkinter for GUI development
 - csv for data export
 - matplotlib for visualizing traffic patterns

5.2 Features Implemented

a. Packet Capture

Using the Scapy library, the application captures live network packets and displays them in a user-friendly GUI.

b. Real-Time Analysis

- Continuous capture mode to analyze packets as they arrive.
- Real-time graphs showing packet rates and protocol distributions.
- Traffic flow analysis to track communication between specific IP addresses.

c. Deep Packet Inspection (DPI)

- Protocol dissection for HTTP/2, SSL/TLS, and other protocols.
- Payload extraction for displaying HTTP headers, HTML content, and more.
- Alerting system for detecting suspicious patterns like port scanning and DDoS attacks.

d. Export Functionality

Allows users to export captured packet details to a CSV file for offline analysis.

e. Enhanced GUI

- Filter options to refine packet capture based on protocols and IP addresses.
- Detailed view of selected packets displayed in a tabular format.
- Clear button to reset the interface.

5.3 Workflow

1. Start packet capture.
2. Display captured packets in real-time.
3. Analyze packets for detailed information and detect anomalies.
4. Provide a detailed view of selected packets.
5. Export packet data for future analysis.
6. Stop packet capture when needed.

6. Results The Network Packet Sniffer successfully captures and analyzes packets in real time. Users can inspect protocol-specific details and identify potential security issues through alerts. Export functionality and graphical visualizations further enhance the application's utility.

7. Challenges and Solutions

- **Challenge:** Parsing advanced protocols like HTTP/2.

- **Solution:** Leveraged additional libraries and raw packet dissection techniques for detailed analysis.
- **Challenge:** Detecting suspicious patterns in real-time.
 - **Solution:** Implemented heuristic algorithms for pattern detection.
- **Challenge:** Ensuring a user-friendly interface.
 - **Solution:** Iterative design improvements based on user feedback.

8. Future Enhancements

- Implement machine learning models for more accurate anomaly detection.
- Expand protocol support to include FTP, SMTP, and DNS.
- Add a cloud-based storage option for exporting packet data.
- Integrate multi-threading for improved performance in high-traffic scenarios.

9. Screenshots

The screenshot displays the 'Enhanced Network Packet Sniffer' application. The top section features a control panel with a 'Protocol Filter' set to 'tcp', a 'Packet Limit' of '10', and buttons for 'Start Sniffing', 'Stop Sniffing', 'Search', 'Save as PCAP', and 'Clear Packets'. Below this is a table listing 10 captured packets with columns for 'No', 'Source (IP + Location)', 'Destination (IP + Location)', and 'Protocol'. The bottom section shows a 'Packet Details' window for the selected packet, displaying fields such as 'Summary', 'Ethernet Source MAC', 'Ethernet Destination MAC', 'Ethernet Type', 'IP Source', 'IP Destination', 'IP Protocol', 'IP TTL (Time To Live)', 'TCP Source Port', 'TCP Destination Port', 'TCP Sequence Number', 'TCP Acknowledgment Number', 'TCP Flags', and 'Payload Size (bytes)'. At the bottom of the details window are buttons for 'Export to CSV' and 'Close'.

No	Source (IP + Location)	Destination (IP + Location)	Protocol
1	192.168.1.11 (Error Fetching Location)	64.233.184.188 (Turlock, United State)	TCP
2	64.233.184.188 (Turlock, United State)	192.168.1.11 (Error Fetching Location)	TCP
3	192.168.1.11 (Error Fetching Location)	208.95.112.1 (Ashburn, United States)	TCP
4	192.168.1.11 (Error Fetching Location)	20.192.44.68 (Pune, India)	TCP
5	208.95.112.1 (Ashburn, United States)	192.168.1.11 (Error Fetching Location)	TCP
6	192.168.1.11 (Error Fetching Location)	208.95.112.1 (Ashburn, United States)	TCP
7	192.168.1.11 (Error Fetching Location)	208.95.112.1 (Ashburn, United States)	TCP
8	20.192.44.68 (Pune, India)	192.168.1.11 (Error Fetching Location)	TCP
9	208.95.112.1 (Ashburn, United States)	192.168.1.11 (Error Fetching Location)	TCP
10	208.95.112.1 (Ashburn, United States)	192.168.1.11 (Error Fetching Location)	TCP

Field	Value
Summary	Ether / IP / TCP 192.168.1.11:52113 > 64.233.184.188:5228 A / Raw
Ethernet Source MAC	b4:8c:9d:16:3f:db
Ethernet Destination MAC	b4:e4:6b:3e:6b:78
Ethernet Type	2048
IP Source	192.168.1.11
IP Destination	64.233.184.188
IP Protocol	6
IP TTL (Time To Live)	128
TCP Source Port	52113
TCP Destination Port	5228
TCP Sequence Number	3660475261
TCP Acknowledgment Number	4268324013
TCP Flags	A
Payload Size (bytes)	41

Export to CSV

Close

10. References

- Scapy documentation: <https://scapy.readthedocs.io>
- Python Tkinter documentation: <https://docs.python.org/3/library/tkinter.html>
- Protocol specifications for HTTP/2 and SSL/TLS: IETF RFCs